

```

# Install required libraries
!pip install pandas numpy matplotlib seaborn scikit-learn statsmodels
scipy --quiet

# Import libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import mean_squared_error, mean_absolute_error,
r2_score, accuracy_score, precision_score, recall_score, f1_score
import statsmodels.api as sm
from statsmodels.tsa.arima.model import ARIMA
from scipy.stats import norm

# Load dataset (Upload gold_price.csv)
from google.colab import files
uploaded = files.upload()

df = pd.read_csv('gold_price.csv')

# Preview data
df['Date'] = pd.to_datetime(df['Date'])
df = df.set_index('Date')
df = df.sort_index()

# Handle missing values
df = df.ffill()

# Feature Engineering
df['Log_Returns'] = np.log(df['Price'] / df['Price'].shift(1))
df['Volatility'] = df['Log_Returns'].rolling(window=30).std()
df['MA50'] = df['Price'].rolling(window=50).mean()

# Prepare features and target together
df['Target'] = df['Price'].shift(-30) # Predict 30 days ahead
df_model = df[['Price', 'MA50', 'Volatility', 'Target']].dropna()

X = df_model[['Price', 'MA50', 'Volatility']]
y = df_model['Target']

# -----
# 1. Stock Price Prediction
# -----

# Linear Regression Model
model_lr = LinearRegression()
model_lr.fit(X, y)

```

```

# Predict
y_pred = model_lr.predict(X)

# Evaluation
print("\nLinear Regression Metrics:")
print("RMSE:", np.sqrt(mean_squared_error(y, y_pred)))
print("MAE:", mean_absolute_error(y, y_pred))
print("R2 Score:", r2_score(y, y_pred))

# Plot Actual vs Predicted
plt.figure(figsize=(10,6))
plt.plot(df_model.index, y, label='Actual')
plt.plot(df_model.index, y_pred, label='Predicted')
plt.legend()
plt.title('Gold Price Prediction - Linear Regression')
plt.show()

# Using ARIMA
model_arima = ARIMA(df['Price'], order=(5,1,0))
model_arima_fit = model_arima.fit()
forecast = model_arima_fit.forecast(steps=30)

plt.figure(figsize=(10,6))
plt.plot(df.index, df['Price'], label='History')
plt.plot(pd.date_range(df.index[-1], periods=30, freq='D'), forecast,
label='ARIMA Forecast')
plt.legend()
plt.title('Gold Price Forecast - ARIMA')
plt.show()

# -----
# 2. Mock Credit Risk Model
# -----

# Mock dataset creation
np.random.seed(0)
mock_credit_data = pd.DataFrame({
    'income': np.random.normal(50000, 15000, 1000),
    'loan_amount': np.random.normal(20000, 5000, 1000),
    'default': np.random.choice([0,1], size=1000)
})

X_credit = mock_credit_data[['income', 'loan_amount']]
y_credit = mock_credit_data['default']

# Model
model_credit = LogisticRegression()
model_credit.fit(X_credit, y_credit)

```

```

y_pred_credit = model_credit.predict(X_credit)

# Evaluation
print("\nCredit Risk Metrics:")
print("Accuracy:", accuracy_score(y_credit, y_pred_credit))
print("Precision:", precision_score(y_credit, y_pred_credit))
print("Recall:", recall_score(y_credit, y_pred_credit))
print("F1 Score:", f1_score(y_credit, y_pred_credit))

# -----
# 3. Mock Revenue/Expense Forecast
# -----

# Fake revenue dataset
months = pd.date_range(start='2019-01-01', periods=60, freq='M')
revenue = 100000 + np.random.normal(0, 10000, 60).cumsum()

df_revenue = pd.DataFrame({'Date': months, 'Revenue': revenue})
df_revenue['Month'] = df_revenue['Date'].dt.month

X_rev = df_revenue[['Month']]
y_rev = df_revenue['Revenue']

model_rev = LinearRegression()
model_rev.fit(X_rev, y_rev)

predicted_rev = model_rev.predict(X_rev)

plt.figure(figsize=(10,6))
plt.plot(df_revenue['Date'], df_revenue['Revenue'], label='Actual')
plt.plot(df_revenue['Date'], predicted_rev, label='Predicted')
plt.legend()
plt.title('Revenue Forecast')
plt.show()

# -----
# 4. Exploratory Data Analysis (EDA)
# -----

# Correlation Matrix
plt.figure(figsize=(8,6))
sns.heatmap(df[['Price', 'Log>Returns', 'Volatility', 'MA50']].corr(),
annot=True)
plt.title('Correlation Matrix')
plt.show()

# Price Plot
df['Price'].plot(figsize=(12,6), title='Gold Price Over Time')
plt.show()

```

```

# Returns distribution
sns.histplot(df['Log_Returns'].dropna(), bins=50, kde=True)
plt.title('Log Returns Distribution')
plt.show()

# -----
# 5. Stochastic Processes (Optional Bonus)
# -----

# Geometric Brownian Motion Simulation
T = 1 # 1 year
N = 252 # trading days
dt = T/N
mu = df['Log_Returns'].mean()*252
sigma = df['Log_Returns'].std()*np.sqrt(252)

S0 = df['Price'].iloc[-1]

price_path = [S0]
for _ in range(N):
    S_next = price_path[-1] * np.exp((mu - 0.5 * sigma**2)*dt +
    sigma*np.sqrt(dt)*np.random.normal())
    price_path.append(S_next)

plt.figure(figsize=(10,6))
plt.plot(price_path)
plt.title('Geometric Brownian Motion - Gold Price Simulation')
plt.show()

# -----
# 6. Black-Scholes Option Pricing (Optional)
# -----

# Black-Scholes formula
def black_scholes_call(S, K, T, r, sigma):
    d1 = (np.log(S/K) + (r + 0.5*sigma**2)*T) / (sigma*np.sqrt(T))
    d2 = d1 - sigma*np.sqrt(T)
    call = (S * norm.cdf(d1)) - (K * np.exp(-r*T) * norm.cdf(d2))
    return call

# Example
K = S0 * 1.05
r = 0.01
T = 0.5
sigma = df['Log_Returns'].std()*np.sqrt(252)

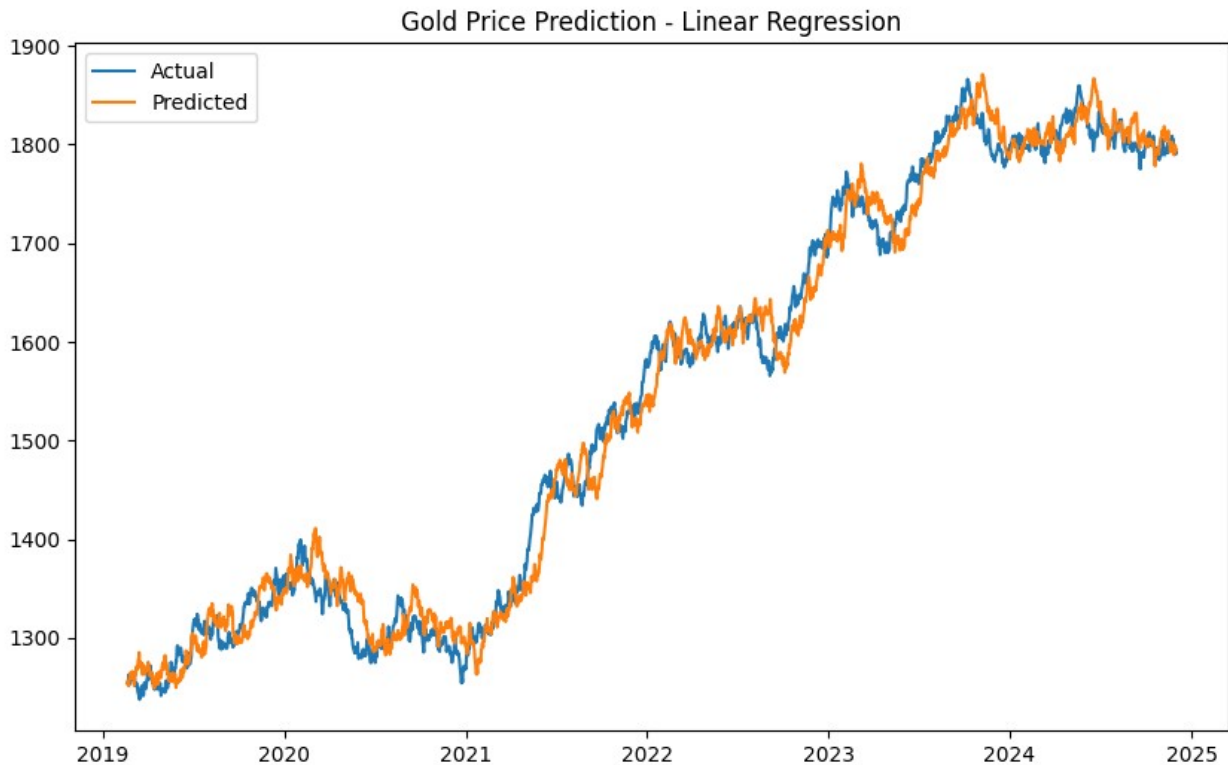
call_price = black_scholes_call(S0, K, T, r, sigma)
print(f"\nBlack-Scholes European Call Option Price: {call_price:.2f}")

<IPython.core.display.HTML object>

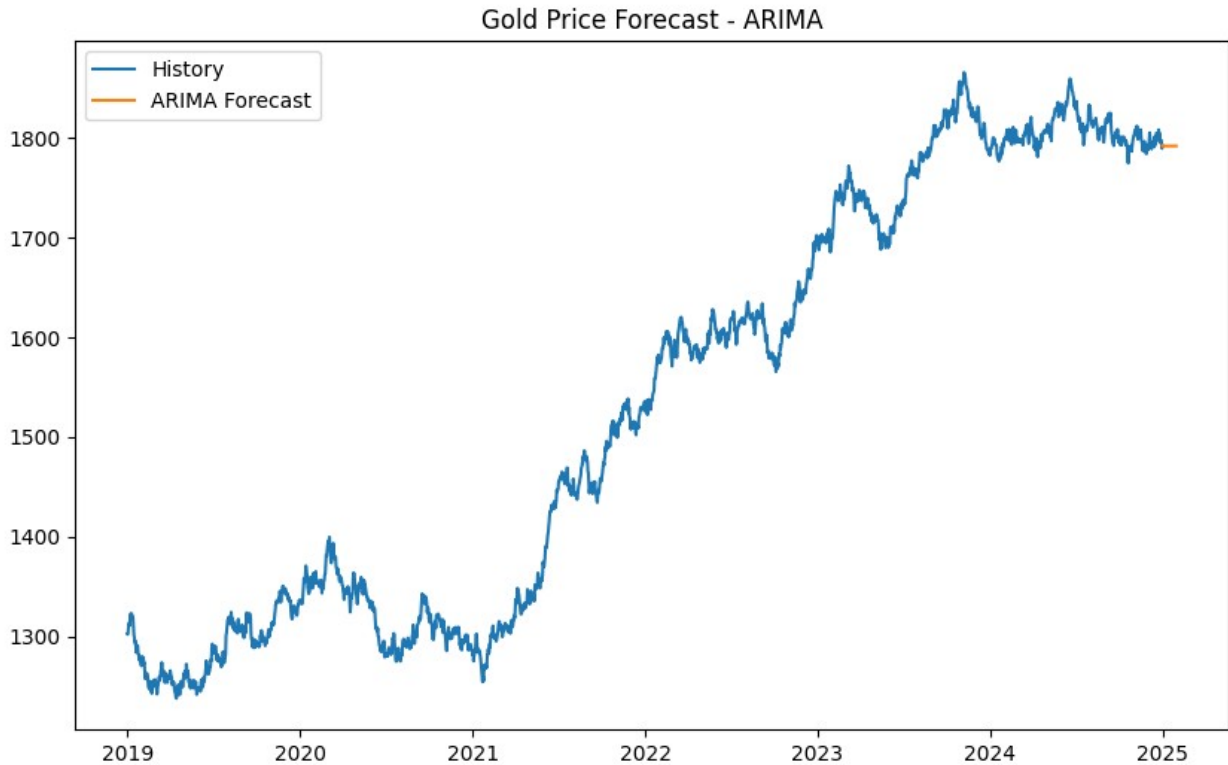
```

Saving gold_price.csv to gold_price (1).csv

Linear Regression Metrics:
RMSE: 25.47757263134937
MAE: 20.471512493920855
 R^2 Score: 0.9850477448616903



```
/usr/local/lib/python3.11/dist-packages/statsmodels/tsa/base/
tsa_model.py:473: ValueWarning: No frequency information was provided,
so inferred frequency D will be used.
    self._init_dates(dates, freq)
/usr/local/lib/python3.11/dist-packages/statsmodels/tsa/base/tsa_model
.py:473: ValueWarning: No frequency information was provided, so
inferred frequency D will be used.
    self._init_dates(dates, freq)
/usr/local/lib/python3.11/dist-packages/statsmodels/tsa/base/tsa_model
.py:473: ValueWarning: No frequency information was provided, so
inferred frequency D will be used.
    self._init_dates(dates, freq)
```



Credit Risk Metrics:

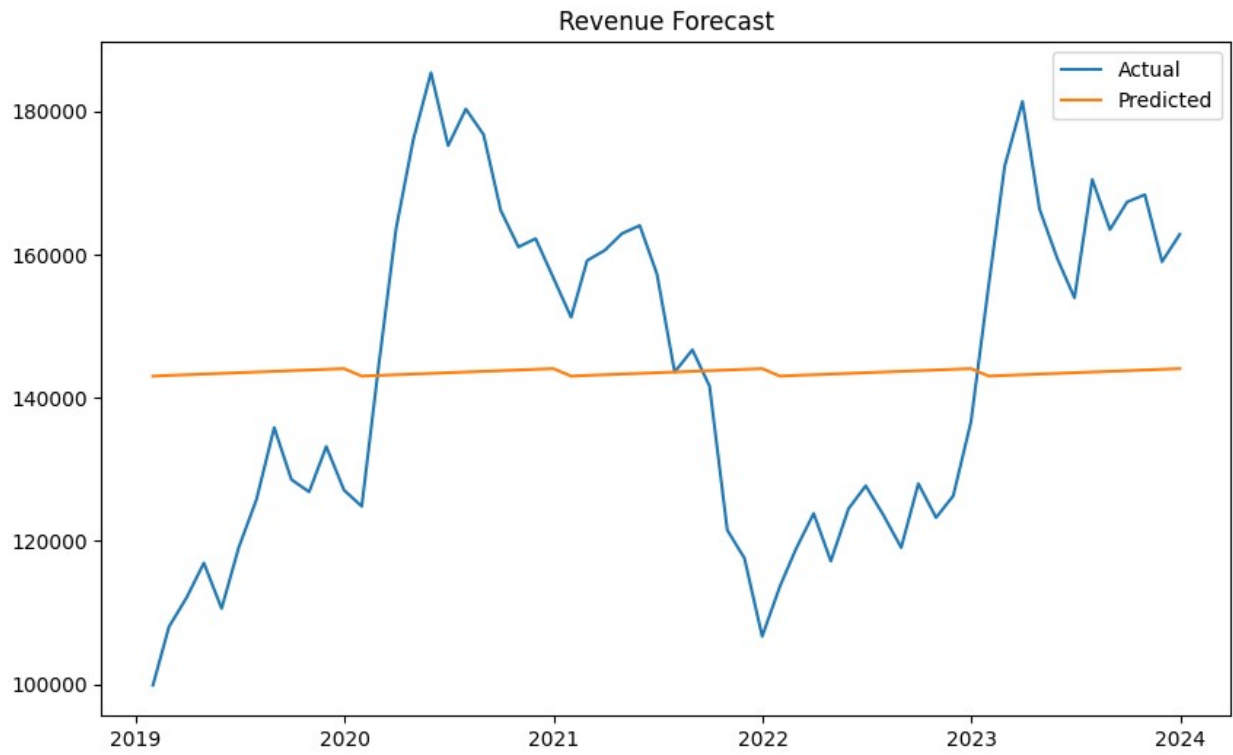
Accuracy: 0.523

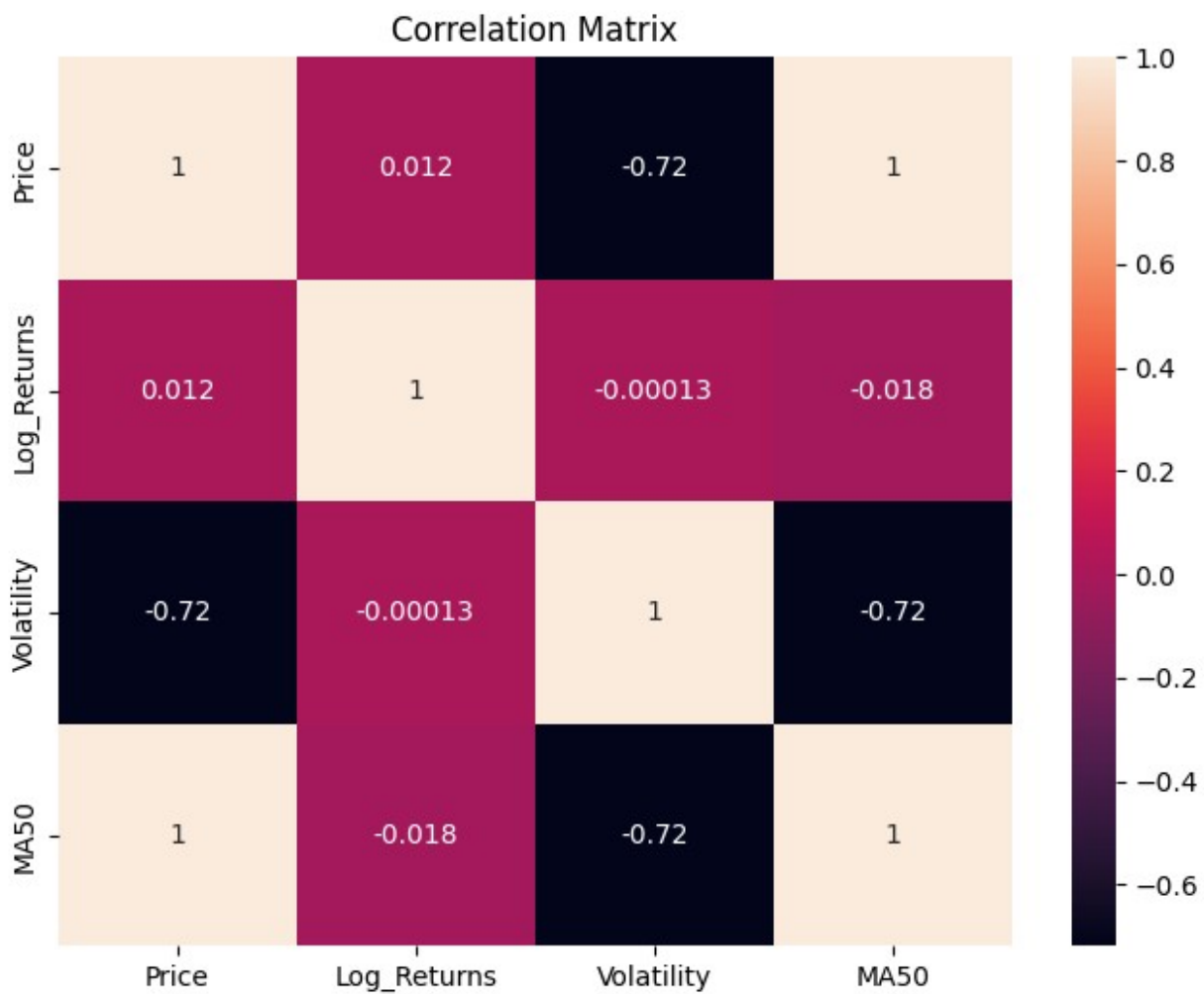
Precision: 0.5218446601941747

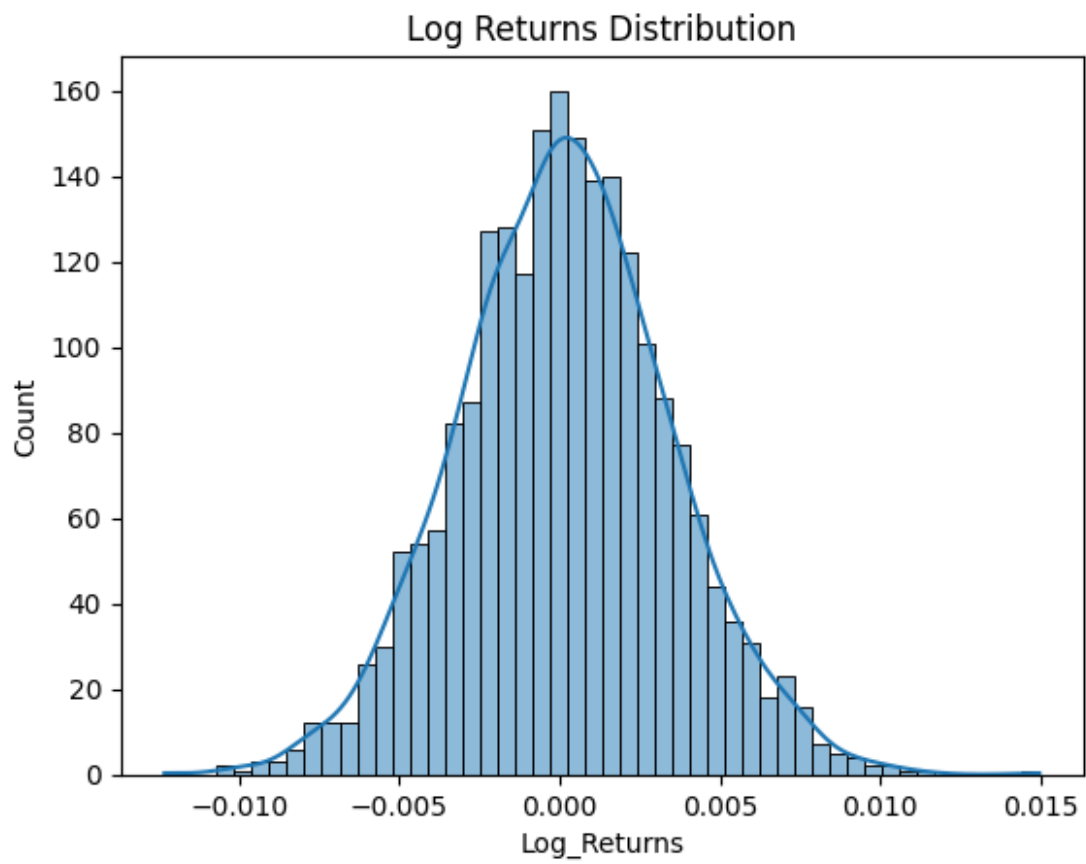
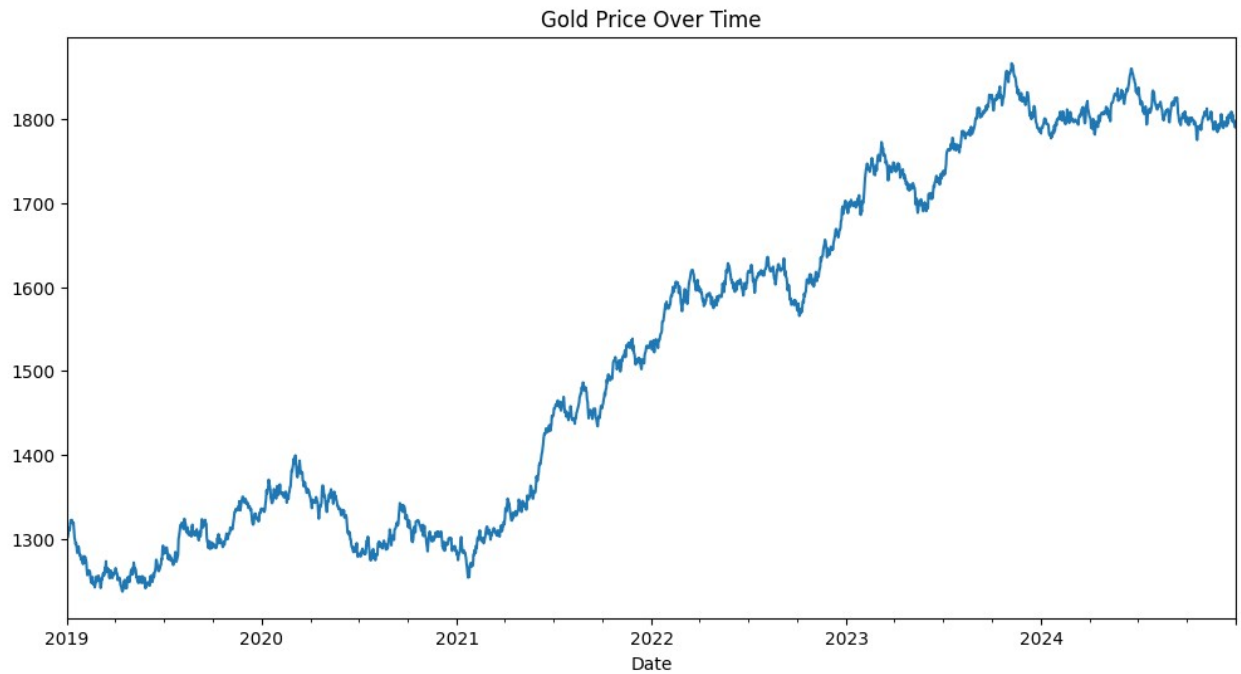
Recall: 0.43434343434343436

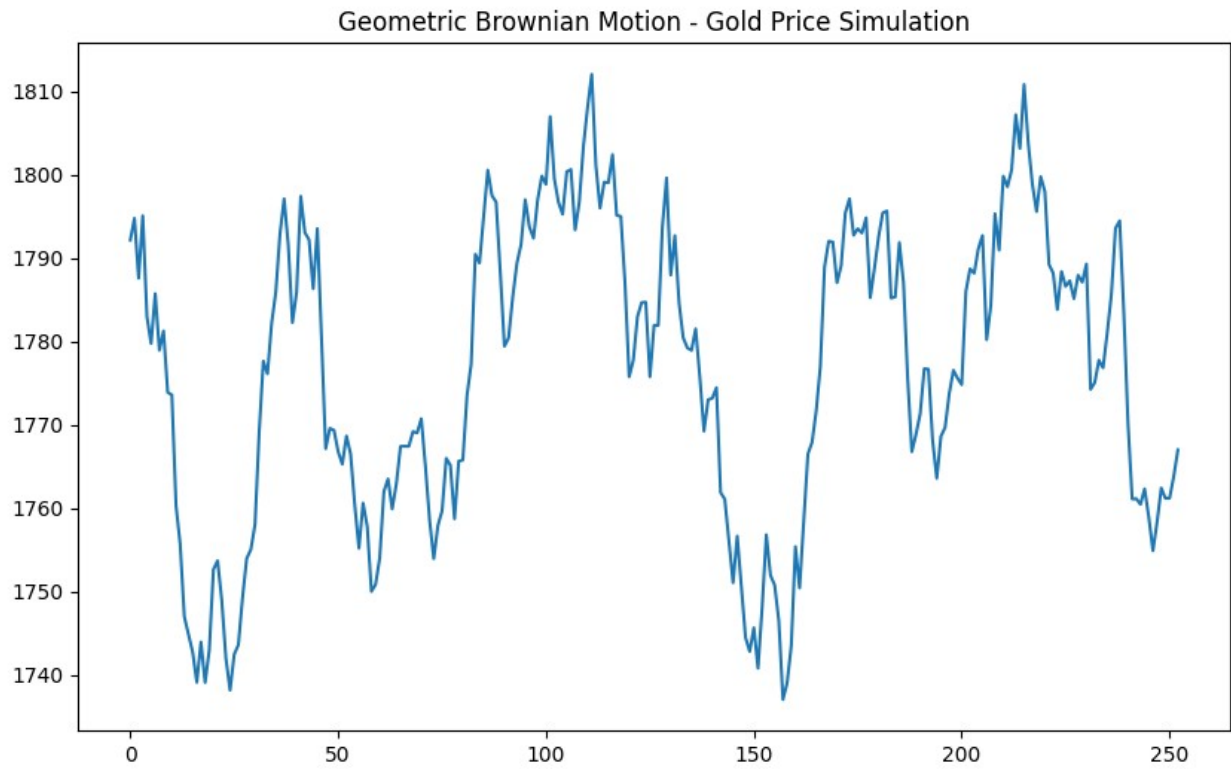
F1 Score: 0.474090407938258

```
<ipython-input-2-d25dffab77a7>:112: FutureWarning: 'M' is deprecated  
and will be removed in a future version, please use 'ME' instead.  
months = pd.date_range(start='2019-01-01', periods=60, freq='M')
```









Black-Scholes European Call Option Price: 3.99